



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт (факультет)
Кафедра

Компьютерных наук
Прикладной математики и системного анализа

ЛАБОРАТОРНАЯ РАБОТА №1

По дисциплине: «Современные платформы разработки ПО».
На тему: «Разработка элементарных программ на языке программирования
Java».

Студент

ПМ-24-1

группа

подпись, дата

Сараев А.П.

фамилия, инициалы

Руководитель

К.Т.Н.

ученая степень, ученое звание

подпись, дата

Немец С.Ю.

фамилия, инициалы

Липецк 2026

Цель работы

Познакомиться с базовыми особенностями языка программирования Java и разработки программ с использованием среды программирования.

Задание

Программа должна предоставлять возможность нескольким пользователям работать с объектами данных, согласно вариантам. Сведения о пользователях должны храниться в отдельном текстовом файле пользователей: для каждого пользователя указывается имя пользователя и пароль. Объекты данных всех пользователей хранятся в одном файле данных.

1. При запуске программы необходимо запросить имя пользователя и его пароль.

2. Если имя пользователя найдено в файле пользователей и введенный пароль верен, то выполняется основная работа программы. В противном случае должно выдаваться сообщение об ошибке и завершение программы.

3. Программа должна предоставлять пользователю следующие возможности по работе с объектами данных:

1. Вывод всех объектов данных пользователя, включая содержательные поля;
2. Добавление нового объекта;
3. Удаление объекта из массива объектов пользователя;
4. Редактирование выбранного объекта.

Замечания!

1. Основная программа должна обрабатывать исключительные ситуации, которые дополнительно фиксируются в текстовом файле.
2. Необходимо разработать механизм сохранения объектов в файл данных, а также загрузки сохраненных объектов при инициализации программы.
3. Язык разработки - Java

Вариант № 15:

Ноутбук {Модель, техн. характеристики, стоимость}.

Текст программы

Main.java

```
package org.example;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.example.models.Laptop;
import org.example.models.User;
import org.example.services.AuthService;
import org.example.services.LaptopService;
import org.example.utils.FileManager;
import java.util.*;

public class Main {
    private static final Logger logger = LogManager.getLogger(Main.class);

    private static boolean printLaptops(String username, LaptopService laptopService) {
        List<Laptop> userLaptops = laptopService.getUserLaptops(username);
        if (userLaptops.isEmpty()) {
            System.out.println("You have 0 laptops");
            return false;
        }
        for (int i = 0; i < userLaptops.size(); i++) {
            System.out.println((i + 1) + ". " + userLaptops.get(i));
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        FileManager fileManager = new FileManager();
        Map<String, User> users = fileManager.loadUsers();
        List<Laptop> laptops = fileManager.loadLaptops();
        AuthService authService = new AuthService(users);
        LaptopService laptopService = new LaptopService(laptops);

        System.out.print("Username: ");
        String username = sc.nextLine();
        System.out.print("Password: ");
        String password = sc.nextLine();

        if (!authService.login(username, password)) {
            System.out.println("Authentication error");
            return;
        }

        while (true) {
            int choice;
            while (true) {
                try {
                    System.out.println("\nChoose:");
                    System.out.println("1 - Show laptops");
                    System.out.println("2 - Add laptop");
                    System.out.println("3 - Delete laptop");
                    System.out.println("4 - Edit laptop");
                    System.out.println("5 - Exit");

                    choice = Integer.parseInt(sc.nextLine());
                } catch (Exception e) {
                    System.out.println("Invalid input. Please enter a number from 1 to 5.");
                }
            }
        }
    }
}
```

```

        break;

    }

    catch (NumberFormatException e) {
        logger.error("Invalid option", e);
    }
}

switch (choice) {
    case 1:
        printLaptops(username, laptopService);
        break;
    case 2:
        try {
            System.out.print("Model: ");
            String model = sc.nextLine();
            System.out.print("Specifications: ");
            String specs = sc.nextLine();
            System.out.print("Price: ");
            double price = Double.parseDouble(sc.nextLine());
            laptopService.addLaptop(username, model, specs, price);

        }
        catch (NumberFormatException e) {
            logger.error("Invalid price", e);
            System.out.println("Price must be number");
        }
        break;
    case 3:
        if(!printLaptops(username, laptopService)) {
            break;
        }
        System.out.print("Enter laptop number to delete: ");
        try {
            int index = Integer.parseInt(sc.nextLine()) - 1;
            if (!laptopService.deleteLaptop(username, index)) {
                System.out.println("Laptop with this number was not found");
            } else {
                System.out.println("Laptop deleted");
            }
        }
        catch (NumberFormatException e) {
            logger.error("Invalid input", e);
            System.out.println("Invalid input");
        }
        break;
    case 4:
        if(!printLaptops(username, laptopService)) {
            break;
        }
        System.out.print("Enter laptop number to edit: ");
        try {
            int index = Integer.parseInt(sc.nextLine()) - 1;
            System.out.println("New model: ");
            String model = sc.nextLine();
            System.out.print("New specifications: ");
            String specs = sc.nextLine();
            System.out.print("New price: ");
            double price = Double.parseDouble(sc.nextLine());
            if (!laptopService.editLaptop(username, index, model, specs, price)) {
                System.out.println("Laptop with this number was not found");
            } else {

```

```

        System.out.println("Laptop updated");
    }
} catch (NumberFormatException e) {
    logger.error("Invalid input", e);
    System.out.println("Invalid input");
}
break;
case 5:
    fileManager.saveLaptops(laptopService.getLaptops());
    System.out.println("Data saved");
    return;
default:
    System.out.println("Invalid option");
}
}
}
}

```

Laptop.java

```

package org.example.models;

import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Laptop {
    private String owner;
    private String model;
    private String specs;
    private double price;

    public Laptop(String owner, String model, String specs, double price) {
        this.owner = owner;
        this.model = model;
        this.specs = specs;
        this.price = price;
    }

    public String toFileString() {
        return owner + ";" + model + ";" + specs + ";" + price;
    }

    @Override
    public String toString() {
        return "Model: " + model + ", Specifications: " + specs + ", Price: " + price;
    }
}

```

User.java

```

package org.example.models;

import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class User {

```

```

private String username;
private String password;

public User(String username, String password) {
    this.username = username;
    this.password = password;
}

public String toFileString() {
    return username + ";" + password;
}
}

```

AuthService.java

```

package org.example.services;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.example.models.User;

import java.util.Map;

public class AuthService {
    private Map<String, User> users;
    private static final Logger logger = LogManager.getLogger(AuthService.class);

    public AuthService(Map<String, User> users) {
        this.users = users;
    }

    public boolean login(String username, String password) {
        User user = users.get(username);
        if (user == null)
            return false;
        return user.getPassword().equals(password);
    }
}

```

LaptopService.java

```

package org.example.services;

import lombok.Getter;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.example.models.Laptop;

import java.util.ArrayList;
import java.util.List;

@Getter
public class LaptopService {
    private List<Laptop> laptops;
    private static final Logger logger = LogManager.getLogger(LaptopService.class);

    public LaptopService(List<Laptop> laptops) {
        this.laptops = new ArrayList<>(laptops);
    }
}

```

```

public List<Laptop> getUserLaptops(String username) {
    List<Laptop> result = new ArrayList<>();
    for (Laptop laptop : laptops) {
        if (laptop.getOwner().equals(username)) {
            result.add(laptop);
        }
    }
    return result;
}

public void addLaptop(String user, String model, String specs, double price) {
    if (model.isEmpty()) {
        logger.error("Model name can't be empty string");
        System.out.println("Model name can't be empty string");
        return;
    }
    if (specs.isEmpty()) {
        logger.error("Specifications can't be empty string");
        System.out.println("Specifications can't be empty string");
        return;
    }
    if (price <= 0) {
        logger.error("Invalid price: " + price);
        System.out.println("Price must be bigger than 0");
        return;
    }
    laptops.add(new Laptop(user, model, specs, price));
}

public boolean deleteLaptop(String user, int index) {
    List<Laptop> userLaptops = getUserLaptops(user);
    if (index < 0 || index >= userLaptops.size()) {
        return false;
    }
    Laptop laptopToDelete = userLaptops.get(index);
    laptops.remove(laptopToDelete);
    return true;
}

public boolean editLaptop(String user, int index, String model, String specs, double price) {
    List<Laptop> userLaptops = getUserLaptops(user);
    if (index < 0 || index >= userLaptops.size()) {
        return false;
    }
    if (model.isEmpty()) {
        logger.error("Model name can't be empty string");
        System.out.println("Model name can't be empty string");
        return false;
    }
    if (specs.isEmpty()) {
        logger.error("Specifications can't be empty string");
        System.out.println("Specifications can't be empty string");
        return false;
    }
    if (price <= 0) {
        logger.error("Invalid price: " + price);
        System.out.println("Price must be bigger than 0");
        return false;
    }
    Laptop laptop = userLaptops.get(index);
    laptop.setModel(model);
    laptop.setSpecs(specs);
}

```

```

        laptop.setPrice(price);
        return true;
    }
}

```

FileManager.java

```

package org.example.utils;

import org.example.models.*;
import org.apache.logging.log4j.*;
import java.io.*;
import java.util.*;

public class FileManager {
    private static final Logger logger = LogManager.getLogger(FileManager.class);

    public Map<String, User> loadUsers() {
        Map<String, User> users = new HashMap<>();
        try (BufferedReader br = new BufferedReader(new FileReader("users.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] p = line.split(";");
                User user = new User(p[0], p[1]);
                users.put(user.getUsername(), user);
            }
        }
        catch (Exception e) {
            logger.error("Error while loading users", e);
        }
        return users;
    }

    public List<Laptop> loadLaptops() {
        List<Laptop> laptops = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader("laptops.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] p = line.split(";");
                laptops.add(new Laptop(p[0], p[1], p[2], Double.parseDouble(p[3])
                ));
            }
        }
        catch (Exception e) {
            logger.error("Error while loading laptops", e);
        }
        return laptops;
    }

    public void saveLaptops(List<Laptop> laptops) {
        try (PrintWriter pw = new PrintWriter(new FileWriter("laptops.txt"))) {
            for (Laptop l : laptops)
                pw.println(l.toString());
        }
        catch (Exception e) {
            logger.error("Error while saving laptops", e);
        }
    }
}

```


Результаты программы на тестовых данных

```
Username: admin
Password: 123

Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Exit
```

Рисунок 1 — авторизация пользователя

```
1
1. Model: wqtre re, Specifications: jdpo ewfwe, Price: 700.0

Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Exit
```

Рисунок 2 — вывод всех объектов

```
2
Model: lenovo
Specifications: i7 13330
Price: 1345

Choose:
1 - Show laptops
2 - Add laptop
3 - Delete laptop
4 - Edit laptop
5 - Exit
1
1. Model: wqtre re, Specifications: jdpo ewfwe, Price: 700.0
2. Model: lenovo, Specifications: i7 13330, Price: 1345.0
```

Рисунок 3 — добавление нового объекта

```

1 2026-03-14 08:28:27 ERROR org.example.utils.FileManager - Error loading users
2 java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
3     at org.example.utils.FileManager.loadUsers(FileManager.java:17) [main/?:?]
4     at org.example.Main.main(Main.java:19) [main/?:?]
5 2026-03-14 09:17:09 ERROR org.example.Main - Неправильный пункт меню
6 java.lang.NumberFormatException: For input string: ""
7     at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67) ~[?:?]
8     at java.base/java.lang.Integer.parseInt(Integer.java:565) ~[?:?]
9     at java.base/java.lang.Integer.parseInt(Integer.java:685) ~[?:?]
10    at org.example.Main.main(Main.java:29) [main/?:?]
11 2026-03-14 10:06:16 ERROR org.example.Main - Invalid price
12 java.lang.NumberFormatException: empty String
13     at java.base/jdk.internal.math.FloatingDecimal.readJavaFormatString(FloatingDecimal.java:1842) ~[?:?]
14     at java.base/jdk.internal.math.FloatingDecimal.parseDouble(FloatingDecimal.java:110) ~[?:?]
15     at java.base/java.lang.Double.parseDouble(Double.java:944) ~[?:?]
16     at org.example.Main.main(Main.java:76) [main/?:?]

```

Рисунок 4 — файл с ошибками

Вывод

В ходе работы разработано приложение с аутентификацией пользователей. Реализованы хранение данных в файлах, операции с объектами и логирование исключений. Программа обеспечивает разграничение доступа пользователей к их данным.